MDPI

Article

HARK: Authority-Typed Review Knowledge for Evidence-Bearing Document Review

Zhitong Yang [1], Kai Wang [1], Qingxian Wang [1], Qi Li [1], Lin Song [1], Dahai Jin [1], Hong Wang [1]

[1] State Grid Customer Service Center, Tianjin 300300, China
* Correspondence: yyyyyyzt@tju.edu.cn

Abstract

Rule-governed document review requires explicit criteria, source-addressable evidence, and a distinction between provisional findings and accountable human disposition. We present HARK, an architecture and an executed system that represent review criteria as rules with hard, hybrid, or soft operational authority. Deterministic verifiers produce checks and candidate spans; a positional evidence gate checks submitted quotations; a policy derives suggested verdicts; and decision records support feedback storage. Technical solution proposals are the sole empirical instantiation. On a fixed developmental regression corpus of 100 synthetic proposals with 253 planted defects, the combined configuration achieves micro-F1 0.798 versus 0.222 for a keyword baseline and 85% policy-verdict agreement. Of 70 policy-non-Pass proposals, 15 are under-called and 10 receive Pass, including four of 23 expected Fail proposals. Sixty-eight of 85 misses enter pending review, without evidence of human resolution. Five development-team runs on three internal proposals demonstrate workflow and persistence, not independent review quality or learning gains. The evaluated snapshot does not enforce authenticated human approval, a complete effective-authority ceiling, or content-addressed configuration recovery. These remain target contracts rather than implemented guarantees. The contribution is an executable review architecture with bounded comparative results and explicit implementation boundaries.

Keywords: document review; review knowledge representation; operational authority; evidence traceability; deterministic verification; human disposition; feedback persistence

1. Introduction

Rule-governed, evidence-bearing document review combines three conditions: explicit review criteria, a source-addressable document, and a distinction between provisional findings and accountable human disposition. Checklist-based inspection provides a foundation for this task [1,2]. The computational problem is not only to generate comments, but to represent which criteria a system can check, what evidence it can supply, and which decisions remain with a reviewer. We study technical solution proposals for a large utility's online service platform: documents describing intended architecture, interfaces, data flows, implementation plans, and risks before implementation. Assurance cases, compliance dossiers, and architecture designs are potential applications of the same task formulation, not domains validated in this study. Explicit review criteria also arise outside internal engineering practice: the EU Artificial Intelligence Act, for example, conditions market access for high-risk systems on documented risk management, technical documentation and conformity assessment [3], which makes the inspectability of review criteria a regulatory as well as an engineering concern.

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Computers* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

The motivating prototype represented 24 expert rules through keyword lists and prompts, without a persistent feedback layer. This arrangement made it difficult to distinguish a requirement's mention from its satisfaction or to connect a proposed finding to an inspectable decision path. Its keyword component is represented by baseline B0 in the evaluation. On the fixed synthetic corpus, B0 recovers 18.6% of planted defects, produces 124 false findings against 47 true findings, and achieves policy-verdict agreement of $\kappa = -0.043$. These results motivate a different representation and routing architecture; they do not identify model failure or isolate the causal contribution of typing.

Consider the criterion that a proposal specify load testing before go-live. "No load test is planned" and "Load testing plan: to be determined" mention the activity without supplying an adequate plan. Conversely, "A pre-production capacity drill will measure response time at 500 TPS before go-live" may provide relevant evidence without the phrase "load test". These illustrative formulations separate lexical presence, document location, and substantive satisfaction. A deterministic cue can support a limited check or identify a passage for review; it does not by itself establish that a testing plan is sufficient.

We call the representation and routing task the *review-knowledge ingestion problem*: how should review criteria, evidence expectations, and decision permissions enter an assisted review system? Keyword checks offer reproducible surface tests; prompts convey contextual instructions; retrieval supplies relevant prior material. None of these mechanisms alone specifies the complete contract linking a criterion, a document range, a proposed finding, and its disposition. HARK combines them through explicit rule records and review states rather than treating the model's output as the final decision.

Two design tensions guide the architecture. First, review criteria mix observable structure with contextual judgement: section presence can be checked mechanically, whereas risk adequacy requires interpretation. Verifiability is therefore relevant to routing [4,5], but the ability to execute a lexical test must not be confused with complete decidability of its business requirement. Second, useful decision history may inform later reviews, yet recording a decision, authorising a knowledge change, and demonstrating improved review quality are different accomplishments. A system must expose these boundaries rather than infer human approval from a name or infer learning benefits from stored feedback.

We present HARK, which retains the prototype name without implying a verified human-gating guarantee, and examine its design, its evaluated implementation snapshot, and its validation boundaries. The system is realised as an extension of a general-purpose agent host: a host-independent review engine carries the representation and decision logic, and a thin host integration supplies storage, tools and delivery, which is what makes the architecture reusable beyond a single deployment. The paper makes four contributions:

(1) A task formulation for rule-governed, evidence-bearing document review and a rule representation that separates evidence production, provisional findings, suggested verdicts, and human disposition (Section 3.1).
(2) An operational authority model with hard, hybrid, and soft routes, verifier-produced absence records, quotation checks, and a deterministic verdict policy, together with the limits of those routes in the evaluated snapshot (Sections 3.2–3.5).
(3) A host-independent engine with a thin deployment integration, carrying feedback records, overrides, exemplars and review-state storage, and distinguished from the target contracts for authenticated approval, complete effective-authority control, and content-addressed recovery (Sections 3.6 and 3.8).
(4) A fixed synthetic end-to-end regression comparison, a traceability and pending-workload analysis, and development-team workflow evidence from three internal proposals in five runs (Section 4).

https://doi.org/10.3390/computers1010000

Four research questions organise the evaluation. **RQ1:** How are review knowledge and operational authority represented, and where are the implementation boundaries? **RQ2:** What detection and verdict trade-offs do the end-to-end configurations exhibit on the fixed synthetic corpus? **RQ3:** What traceability and gate-acceptance evidence is available, and what unresolved workload is routed to human review? **RQ4:** What evidence supports decision persistence, human approval, and version recovery, and which gaps remain? RQ2 concerns a combined-configuration comparison, not the causal effect of typing; RQ4 concerns workflow and recovery evidence, not online improvement.

Figure 1 outlines the architecture. A Markdown document is parsed into source-addressable structures; verifier checks produce findings or candidate spans; a judge submits quotations; a gate checks their location; and a deterministic policy derives a suggested verdict from eligible findings. Decisions and feedback are stored separately. In the evaluated snapshot, the gate tests candidate overlap rather than strict containment, gate-confirmed findings can affect the recommendation before human adoption, and the final-verdict tool requires a reviewer string rather than authenticated approval. The stricter separation of authority and complete configuration recovery described as target contracts are not implemented guarantees.

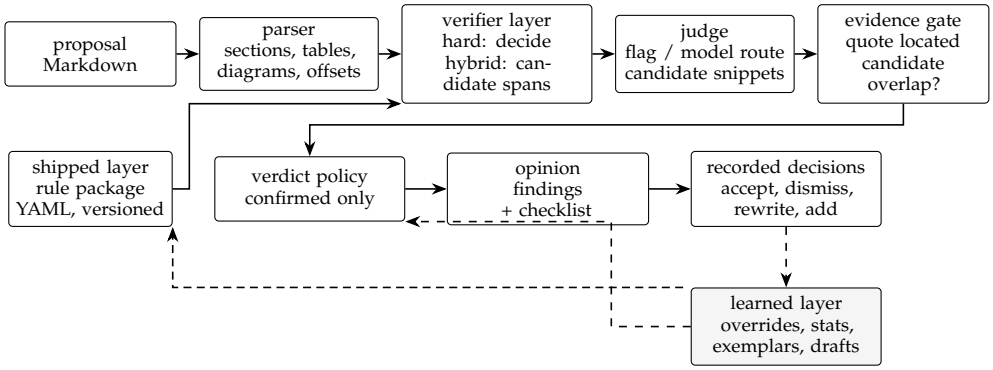


Figure 1. Review architecture and intended feedback flow. Solid arrows depict document processing; dashed arrows depict decision and knowledge-update paths. The diagram summarises the design, not a verified security boundary. The evaluated gate checks quotation location and candidate overlap; tool-entered reviewer names and instruction quotations do not authenticate human approval. Controls covering all authority-changing operations and immutable effective-configuration snapshots remain target contracts.

2. Related Work

Rule-governed, evidence-bearing document review.

Document inspection has a long history of using explicit review procedures and checklists [1,2]. Requirements engineering adds quality criteria, smell detection, and traceability to this tradition [4–6]. We use *rule-governed, evidence-bearing document review* to denote a task in which an organisation supplies review requirements, a system associates proposed findings with evidence in the submitted material, and an accountable reviewer resolves issues that the automated procedure cannot settle. Requirements specifications, policy-governed contracts, and technical solution proposals provide different instances of this task [7–9]. Security requirements and industrial certification research further illustrate the importance of explicit review criteria [10,11], while traceability has also been studied for an in-the-wild augmented-reality learning service [12]. These sources motivate the broader task setting; their existence does not establish that HARK transfers across those domains. Technical solution proposal review remains the only empirical case in this study.

Requirements quality and operational verifiability.

Requirements-smell analysis detects cues such as weak wording, placeholders, and missing units [6]. Veizaga et al. combine smell detection with recommendations based on pre-trained language models [13], and AirReq applies smell detection and elimination to commercial aircraft system requirements [14]. HARK draws on these techniques for structural checks and candidate localisation, but a surface cue is not by itself a complete assessment of whether a requirement is satisfied. A further distinction concerns the meaning of *verifiability*. As a requirements-quality attribute, verifiability concerns whether satisfaction can be established through an appropriate verification procedure [4,5]; it does not mean that the current document parser can decide the requirement. HARK's tiers are operational assignments relative to the available parser, verifier, representation, and rule package. A requirement requiring measurements, external records, or an image may be verifiable in principle while remaining outside this prototype's deterministic checking capability. The tier therefore specifies output eligibility within an implementation, not an intrinsic quality ranking of the requirement.

LLM-assisted requirements review and evaluation.

Reviews of LLM applications in requirements engineering describe uses across extraction, analysis, and specification [15]; a dual-perspective review distinguishes using LLMs for code verification from verifying their generated outputs [16]. MARV uses multiple role agents and retrieval of standards for software requirements specification validation [7]. ReqPlan-Eval explicitly couples evidence-aware routing with human requirements-to-planning decisions and discloses the limits of its evaluation [8]. These approaches supply relevant precedents for evidence use and reviewer involvement. The inspected MARV material does not establish the detailed behaviour of document-range validation or persistent feedback, so we do not infer that those capabilities are absent.

Hallucination research motivates checking generated claims [17], but its findings should not be converted into a universal claim that models cannot perform deterministic tasks. Long-context position effects [18], limits of intrinsic reasoning self-correction without external feedback [19], and model-contingent polarity in a hotel-review annotation task [20] concern distinct failure modes and settings. They motivate separating model judgement from permission to issue a result, rather than proving the effectiveness of HARK's particular routing policy. Likewise, the evaluation literature [21] motivates reporting the corpus, judge, and task conditions under which a mechanism was tested. HARK's deterministic-judge evaluation addresses this bounded mechanism question, not the capabilities of arbitrary deployed LLMs.

Neuro-symbolic policy review.

Neuro-symbolic integration offers several ways for symbolic components to constrain learned components [22]. PolicyGuard compiles organisational policies into typed relational rules, uses LLM extraction to ground atoms from evidence, and applies symbolic rule evaluation [9]. Its v1 report also documents legal-expert refinement during review-engine construction and validation: Appendix G describes editing extraction questions, correcting rule-combination logic, and treating a triggered finding as a case-specific exception while preserving the audit trail. It is therefore inaccurate to characterise PolicyGuard as having no feedback or human maintenance. The comparison instead concerns the role of the evidence-producing component and the lifecycle of review knowledge: PolicyGuard reports atom grounding, symbolic evaluation, and local expert corrections; HARK studies rule-specific eligibility for automatic findings and a stored review-feedback layer. These

are different architectural choices, not evidence of a general deficit in PolicyGuard or proof that HARK has implemented complete human-controlled evolution.

Galitsky and Rybalov study symbolic checking of LLM-generated fault explanations and control actions against physical and sensor constraints in *process control* [23]. That setting illustrates constraint-based rejection, but it is not a study of design-document review. Nor do these examples justify classifying all neuro-symbolic systems as assigning the same authority to every learned output. HARK's relevant design choice is to make the eligible output and its evidence conditions explicit for each rule before reviewing a document.

Industrial review and editable feedback knowledge.

Code review provides adjacent evidence about learning from reviewer reactions, although its artefacts and evaluation units differ from document review. Pornprasit and Tantithamthavorn compare fine-tuning and prompting for code review automation [24]. BitsAI-CR's v1 architecture contains *two fine-tuned LLMs*, RuleChecker and ReviewFilter; RuleChecker is not a deterministic rule engine. Its data flywheel updates review rules and their taxonomy as well as supplying data for model improvement [25]. The RovoDev report evaluates a deployed reviewer using offline and online measurements and developer feedback [26]. Lin et al. analyse developer replies to CodeRabbit reviews, reporting rejection and misalignment within the reply-bearing sample rather than establishing rejection rates for all generated comments [27].

Feedback is not necessarily hidden in model weights or inaccessible to users. CodeRabbit's official Learnings documentation, as accessed on 5 November 2026, describes stored user-generated learnings that can be viewed and edited, with role-governed deletion through the interface and removal requests through chat [28]. These controls are a positive precedent for inspectable review knowledge. They do not, by themselves, establish complete historical rollback, permission-ceiling enforcement, or replay of the exact knowledge content used for an earlier review. Conversely, those capabilities cannot be declared absent merely because an industrial evaluation paper did not assess them. HARK's stored feedback should be compared with these concrete mechanisms, not with a homogeneous category of supposedly opaque industrial agents.

Grounding, non-detection, and pending decisions.

Attributed generation links claims to cited sources [29]. Claim-level evaluation of verbatim citation distinguishes quotation compliance from whether the quoted passage actually supports the claim [30]. HARK adds a rule-associated, tool-side eligibility check: the evaluated prototype locates a submitted quotation and checks overlap with verifier-provided candidate evidence for that rule. This *located-overlap gate* is narrower than a guarantee of complete span containment or semantic entailment. A located quotation can still be irrelevant or misinterpreted, and candidate retrieval can miss relevant material.

Non-detection also needs an explicit evidence boundary. A verifier-owned non-detection record reports that a specified procedure did not find expected material in its declared checking scope; it is not proof that the requirement is unsatisfied in the world, or that relevant content is absent from every representation of the document. HARK reserves such records for verifier output rather than accepting a model-generated assertion of absence. When a hybrid rule lacks usable candidates, or a proposed finding does not meet its evidence conditions, an explicit *pending* state preserves the unresolved obligation instead of turning it into either a pass or a confirmed failure. Documents remain untrusted inputs: prompt-injection attacks against LLM judges [31] motivate this separation, but an evidence gate alone does not establish resistance to injection or protect every write path.

Human control, memory, and version requirements.

Human-in-the-loop research distinguishes learning arrangements according to how control is allocated between system and expert [32]. HARK adopts this control question at the level of review rules and feedback events; the presence of a named reviewer or a stored approval text is not equivalent to authenticating the approving human. Agent-memory research [33], self-refinement [34], reflection-based memory [35], and self-modifying agents [36] show multiple ways to reuse interactions or improve behaviour. Neither prompting nor retrieval inherently precludes audit logs, persistent consolidation, or versioned knowledge. Those properties depend on the surrounding storage, update, and permission mechanisms.

Memory extraction can expose sensitive stored content [37]. HARK's structured feedback records and short exemplars make stored fields explicit, but this representation has not been shown to provide a privacy guarantee. More generally, bounded evolution requires separately establishing the identity and authority of the approving actor, rejecting updates that exceed an authorised permission ceiling, and retaining the actual content needed to replay a prior knowledge state. The evaluated prototype provides operational tier routing, a located-overlap gate, and stored feedback; *complete human authentication, permission-ceiling enforcement, and content replay are not yet implemented*. Consequently, feedback storage and version labels are reported mechanisms, not evidence that every update is human-authorised, reversible, or beneficial over time.

Positioning and contribution boundary.

Table 1 compares concrete reports or explicitly labelled family illustrations, rather than scoring entire research traditions. The positive contribution studied here is a system representation that connects *rule-specific output eligibility*, *verifier-owned non-detection records*, *explicit pending outcomes*, and *feedback/version requirements* within one review workflow. The three tier labels are not themselves a novelty claim. We neither claim a first combination based on unverified negative capabilities of other systems nor claim that the full human-gated lifecycle has been achieved. The measurements reported later concern the implemented proposal-review prototype and its stated judge; broader document classes and strict human-authorised evolution remain outside the demonstrated scope.

3. Materials and Methods

3.1. Task Formulation and Scope

We study *rule-governed, evidence-bearing document review*. An instance consists of a source-addressable document d, an explicit rule package K, a policy P for aggregating findings, and an accountable reviewer. The system prepares rule-linked findings and unresolved items; the reviewer determines their disposition and the final opinion. This task is broader than technical proposal review but narrower than arbitrary document understanding: it requires explicit criteria, identifiable evidence sources and a meaningful distinction between a machine recommendation and an accountable decision. A source range identifies what was read, not whether its contents are true or sufficient.

The empirical instance is preliminary review of Chinese technical solution proposals before construction. Its primary business object is the *adequacy of the proposed design*, not proof that the proposed implementation has already been deployed. A rule must specify the lifecycle stage, applicability, expected evidence and predicate being checked. We retain the released package and labels for the retrospective evaluation, rather than silently relabel them to fit this formulation. Where its predicates conflate design and implementation, or parser support with business compliance, the discrepancy is reported and must be resolved before a revised package is evaluated.

Table 1. Mechanism-level positioning using named reports and versions. "Not established" denotes a limit of the inspected evidence, not an assertion that a system lacks the capability. Rows are not a common-benchmark comparison; reported controls are not treated as proof of complete rollback or human authentication.

Report or illustration	Knowledge and decision mechanism	Evidence and unresolved outcomes	Feedback and demonstrated control boundary
2025 keyword prototype (reimplemented baseline)	Substring checks per proposal rule	Keyword matches; no satisfaction semantics	No persistent feedback in this baseline implementation
Requirements-smell family illustration [6,13,14]	Cue detection; some systems add learned recommendations	Requirement-level quality cues; coverage depends on the analyser	No uniform family-wide lifecycle or rollback claim
MARV, 2025 thesis [7]	Role agents with retrieved standards	Requirements validation; detailed document-range gate not established	Persistent feedback and rollback not established by inspected material
ReqPlan-Eval, 2026 report [8]	Evidence-aware routing; final human decisions	Evidence records and disclosed evaluation limits	Knowledge evolution not reported as a tested mechanism
PolicyGuard, arXiv v1 [9]	Typed relational policy; LLM-grounded atoms; symbolic evaluation	Atom-level evidence and rule traces	Construction/validation expert corrections and audit trail; complete deployed rollback not established
BitsAI-CR, arXiv v1 [25]	Fine-tuned LLM RuleChecker and ReviewFilter	Generated comments and learned secondary validation	Data flywheel refines models and rule taxonomy; per-reviewer rollback not established
RovoDev, arXiv v2 report [26]	Deployed code-review generation and filtering	Comment quality evaluated offline and online	Developer feedback evaluated; full knowledge-history replay not established
CodeRabbit Learnings, official documentation accessed 5 Nov. 2026 [28]	Stored user-generated review knowledge	Knowledge applied to subsequent code reviews	View/edit/delete controls; these alone do not establish complete historical rollback
HARK evaluated prototype	Operational tier routing and rule-specific output eligibility	Located-overlap gate; verifier non-detection; explicit pending	Stored feedback; complete human authentication, permission ceiling, and content replay not yet implemented

Four requirements guide the design: rule-linked evidence and explicit non-detection records; rule-specific eligibility to contribute a finding; a visible distinction between confirmed findings, unresolved items and final disposition; and inspectable feedback with controlled activation and recoverable configuration. These are design requirements, not four completed implementation guarantees. Table 2 separates the evaluated prototype from the stronger deployment contract.

3.2. Operational Authority Types

Each YAML rule declares an identifier, dimension, severity, tier, question, rationale, remediation, applicability condition, verifier specification and status. A shared lexicon supplies heading aliases, placeholder and negation tokens, and keyword families. The package contains 28 rules: 10 hard, 13 hybrid and 5 soft. Seven have applicability preconditions; the lexicon contains 25 keyword families, 11 heading groups, 12 placeholder tokens and 11 negation tokens.

The tier is an *operational routing declaration*. A hard rule uses a decisive implemented predicate; a hybrid rule combines deterministic checks or candidate localisation with a judgement step; a soft rule raises an unresolved item. Decisive execution of a predicate

Table 2. Requirements and evidence status. The evaluated prototype does not yet implement the complete approval and recovery contract.

Requirement	Evaluated prototype	Remaining deployment requirement
Evidence traceability	Rule identifiers, source ranges, verifier absence records and recorded gate outcomes	Separate source location, scope relation and semantic support; validate converted inputs
Rule-specific eligibility	Hard/hybrid/soft routing and package validation	Validate effective authority after every update, not only the tier field
Accountable disposition	Confirmed/pending states, deterministic recommendation and recorded decisions	Host-authenticated approval; explicit disposition or waiver of pending items before signing
Feedback and recovery	Overrides, statistics, exemplars, audit records and an override-count version string	Content-addressed immutable configurations, approval binding and demonstrated restoration

does not prove that the predicate fully captures the business requirement. For example, a lexical declaration of a load-test plan does not establish its scope, responsibilities, timing or performance criteria. Assigning a hard tier to that proxy cannot make those unmeasured properties automatically verifiable. Domain owners must first define the requirement and then specify which subpredicates the verifier actually covers.

Table 3. Default routes in the evaluated package. These routes describe finding eligibility under the frozen configuration, not authentication of human approval.

Tier	Verifier role	Default outcomes	Finding route
hard	Decisive configured predicate	pass, fail, not applicable	Verifier-generated finding; model registration rejected
hybrid	Structural checks and candidate localisation	structural result, candidate, pending, not applicable	Structural failure may be automatic; otherwise a gate-accepted judgement may count
soft	No verifier	unresolved item	Model registration remains pending until an explicit disposition

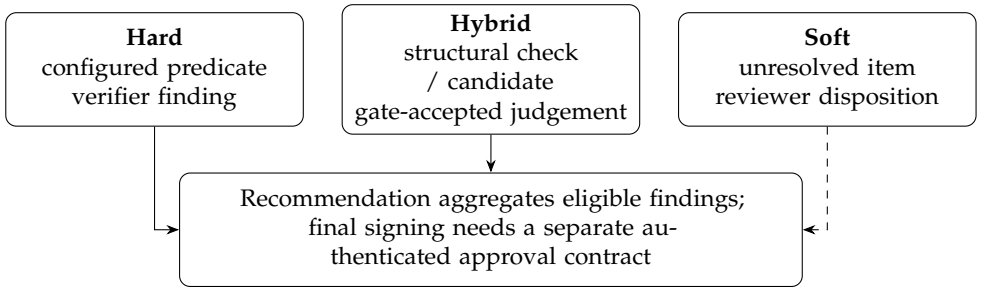


Figure 2. Operational routes and their boundary. A tier is one component of effective authority, not a sufficient permission ceiling for subsequent edits. The complete signing contract is a deployment requirement, not an evaluated capability.

In the default registration path, absence evidence is verifier-owned: a model cannot establish non-detection by inventing a quotation such as "not mentioned". A verifier absence record identifies the expected section and the failed search predicate; it does not establish semantic absence outside the parser's supported input and vocabulary. Hybrid non-detection defaults to pending. Candidate existence is not satisfaction, and lack of a registered finding is not a verified pass.

Package validation rejects inconsistent verifier/tier combinations, missing lexicon references and duplicate identifiers. This is useful static validation but does not enforce

the same restrictions after every learned edit. The evaluated update paths can change no-candidate behaviour, activate a valid hybrid draft, alter severity and reset overrides without preserving a complete effective-authority ceiling. Consequently, immutability of the tier field alone does not establish bounded authority.

All four blocker rules in this package (CAP-001, SEC-001, SEC-002 and SEC-004) are hybrid; the ten hard rules are major or minor. This distribution motivates tier-specific failure analysis, but is a characteristic of this checklist, not a general inverse relationship between severity and verifiability.

Table 4. Composition of the frozen package used in the retrospective evaluation.

Dimension	Hard	Hybrid	Soft	Total	Blocker	Major	Minor
Capacity	2	3	1	6	1	5	0
Security	0	4	1	5	3	2	0
Interface	2	3	0	5	0	3	2
Documentation	6	0	2	8	0	4	4
Channel	0	1	0	1	0	1	0
Data synchronisation	0	1	0	1	0	1	0
Configuration	0	0	1	1	0	0	1
Monitoring	0	1	0	1	0	0	1
Total	10	13	5	28	4	16	8

3.3. Parsing, Verifiers and Business Predicates

The review engine accepts Markdown. Word and PDF conversion occurs upstream through the host's office tooling; no conversion-fidelity evaluation is reported here. The parser constructs a heading tree, tables and Mermaid sequence diagrams [38], with source offsets. Its normalisation includes NFKC folding, punctuation folding, emphasis removal and whitespace collapsing, with a map back to the source. Chinese and Latin sentence terminators and table-cell boundaries define candidate units. Absence records use an expected section and a zero-length source range.

Table 5. Implemented verifier families. An established parser predicate is not necessarily the complete business predicate. Uses count occurrences in verifier trees.

Family	Uses	Implemented check	Scope limitation
statement	16	Keyword matches, local negation, placeholders, assertion cues	Does not establish semantic sufficiency
section	9	Heading, body length and placeholder checks	Alias and format dependent
table	3	Column groups, row count and placeholder cells	Not correctness of cell content
mermaid-sequence	3	Parseable diagram, participants, messages and required families	Cannot inspect arbitrary embedded images
cross-ref	3	Attachment-reference resolution	Not attachment correctness
quantity	1	Number/unit near a context family	Not validity of the quantity
all, first	4, 2	Conjunction and ordered locating fallback	Inherits child limitations

Negation checks use a window around a matched family; placeholder checks combine lexical tokens and structural cues. These choices recover defects that substring presence misses, but their correctness remains relative to the declared predicate and lexicon. The frozen diagram rules report an image-only diagram as a failure because they require Mermaid. That result supports compliance with a *declared machine-readable format*, not a finding that the depicted design is missing or wrong. Unless an organisation requires that format, unsupported image input should become *needs manual review*, rather than business noncompliance. This correction changes rule behaviour and may require label migration

and reevaluation; the retrospective counts below are not presented as results of such a corrected package.

3.4. Located-Evidence Gate

Model findings use a registration tool. The gate rejects hard-rule registration, disallows model-produced absence evidence, checks normalised quote length (8–300 characters), locates the quotation and checks its relation to a hybrid candidate. Unknown or soft-rule submissions remain pending rather than gaining automatic eligibility. Rejection reasons are stored for inspection and possible repair.

Algorithm 1: Gate in the evaluated prototype (simplified). Scope membership is overlap, not complete containment; location does not establish entailment.

Input: document, rule id, evidence list, candidate ranges, optional stated section
Output: gate-confirmed finding or pending item with reasons

```
Resolve rule; unknown or soft rule adds a pending reason
if tier is hard then
    Reject model registration
end if
if evidence is empty then
    Add missing-evidence reason
end if
for each evidence item e do
    if e asserts absence or starts with an absence marker then
        Add absence-not-allowed reason
    end if
    Normalise e.quote; add a reason if length is outside 8–300
    Find all source hits of the normalised quote
    Prefer candidate-and-section hit, then candidate hit, then section hit, then first hit
    if no hit exists then
        Add quote-not-found reason
    else
        Attach the selected source range h
        if tier is hybrid and h overlaps no candidate then
            Add out-of-candidates reason
        end if
    end if
end for
return gate-confirmed if no reason; otherwise pending with reasons
```

The gate establishes recoverable quotation and a positional relationship, not that the quotation supports the allegation. In particular, partial overlap can pass, and a genuine but misleading quotation can be accepted. A strict containment contract would instead require every quoted character to lie within an allowed range and would need separate boundary tests and a new evaluation. We do not attribute zero false positives or prompt-injection resistance to this gate. The reported 42/53 fraction is *gate acceptance*, not semantic evidence validity.

3.5. Recommendation and Final Disposition

Let C contain findings with an eligible route (hard, hybrid or expert) and status confirmed, adopted or rewritten (the accept action sets the adopted state). Under the frozen

policy, gate-confirmed hybrid findings can already enter C before expert adoption. With m the number of minor findings in C and $\theta = 3$,

$$P(C) = \begin{cases} \text{Fail}, & \exists i \in C : \text{severity}(i) = \text{blocker}, \\ \text{Revise}, & \exists i \in C : \text{severity}(i) = \text{major or } m \geq \theta, \\ \text{Pass}, & \text{otherwise.} \end{cases}$$

Pending and soft items do not count; dismissed findings leave C. Opinion-channel findings become numbered clauses; checklist-channel findings form a correction annex, but both eligible channels contribute to the policy. Deterministic aggregation means repeatability for the same finding set; it does not make that set correct, and a model can influence the recommendation through accepted registrations or severity choices.

The prototype records a final verdict through a tool requiring a reviewer-name field. That field is provenance metadata, not authentication: an agent can supply a name. Finalisation also does not reliably require every pending item to have been disposed of. The current implementation therefore supports a *trusted-operator workflow*, not a demonstrated software-enforced human-signature boundary. Suggested/final verdict disagreement may be recorded, but cannot by itself measure rubber-stamping.

3.6. Feedback Records and Configuration Recovery

The review preset prescribes preliminary review, expert disposition and finalisation. It asks the agent to present findings in small batches with quotes and suggested remedies; accept, dismiss, rewrite and add actions become structured records. Statistics and exemplars can subsequently be recalled. The software stores feedback; this is not evidence that later judgements improve, or that the person recorded as operator genuinely approved each action.

Table 6. Storage objects in the prototype. Write paths describe implemented operations, not independently authenticated human acts.

Object	Contents and write operation	Subsequent use
rule_overrides	Status, severity, suggestions, whitelisted parameters, drafts; update tools	Effective rule loading and rules view
reviews	Input digest, recommendations, final value, timings, displayed version	Resumption and dashboard
issues	Findings, evidence, tier, severity and gate outcome; check/registration/decision	Opinion and evaluation
feedback	Accept/dismiss/rewrite records with reasons	Aggregation
exemplars	Accepted suggestions and rewrites; aggregation	Suggestion context
rule_candidates	Unmatched additions and draft rules	Rule inspection/activation
eval_runs	Configuration results and per-rule outputs	Dashboard and retrospective analysis

A requested severity or status edit can carry a verbatim `basis.quote`, recorded with operator, time and review. An arbitrary sufficiently long string satisfies this check; it does not prove consent. Parameter changes and valid-draft activation can also change decision eligibility while leaving the tier field unchanged. Reset removes overrides and keeps statistics, but may restore stronger shipped settings than the last approved settings.

The displayed identifier `pack@ver+ovN` counts rules with active overrides. Two distinct configurations can have the same N; it is not a content identity and does not enable exact historical replay. Audit records are useful provenance, but reversibility requires retaining the actual configuration to be restored and validating its authority, rather than merely deleting overrides.

3.7. Required Human-Approval and Recovery Contract

For deployment under a software-enforced human-approval claim, we specify a stronger contract, not an already implemented extension. Effective authority is a tuple of permitted decision subject, admissible evidence, no-candidate behaviour, severity, applicability, activation status and policy eligibility. Every write path must validate the merged configuration against its approved ceiling, including parameter edits, draft activation, finding severity, enable/disable and reset.

A trusted host must issue an approval event after displaying the exact object and action to an authenticated reviewer. The event must bind review/rule identity, input and opinion/configuration content, action and values, principal and consumption or expiry state. An agent-provided name or quotation must not mint that event. Content changes, cross-object use and replay invalidate approval. Final signing requires explicit disposition of pending items or a separately authorised waiver with a reason; the interface must not confuse "Pass on confirmed findings" with review completion.

Recoverable versions require content-addressed immutable snapshots of rules, lexicon, overrides, policy and relevant parser/verifier versions, linked to input and approval records. Restoration must recover the specified approved state and reproduce its expected behaviour on fixed inputs while retaining the audit history. Acceptance tests must include valid approvals as well as forged, missing, replayed and changed-content approvals; authority-changing parameter/draft/severity/reset paths; unresolved-item signing; partial-overlap quotes; and distinct configurations with the same override count. These tests and the resulting repairs remain future engineering work in this manuscript.

This contract assumes trusted host authentication, storage and the controlled review APIs. A malicious administrator, or an agent with arbitrary host shell/file write access, operates outside the boundary that the review layer can defend. Scoped reading may reduce exposed context, but does not establish protection against prompt injection, semantic manipulation or exfiltration [31,37].

3.8. Implementation and Evaluated Version

HARK [39] is delivered as an extension package for a general-purpose conversational agent host, which supplies model access, file conversion, structured questions, delivery, JSON-backed storage and a web server. Hosting is a deliberate design position rather than an implementation shortcut: a review system has to specify which party may retrieve evidence, which may propose a finding, and which may commit a verdict, and anchoring those permissions at a host boundary keeps them explicit and separable from review logic. The engine itself is host-independent: the parser, verifiers, gate, policy, aggregation and renderer are pure functions over rule packages, documents and recorded state, and the engine modules import nothing from the host, so the review semantics can be executed and audited without a particular deployment. The package defines seven storage objects, nine tools, three state-query slash commands and a host-served dashboard while reusing host authentication, file handling and delivery; it requires no separate application backend or embedding index. That reuse is what keeps the integration surface small, but operation-specific approval remains a separate requirement and is not supplied by a reviewer-name field.

The retrospective analysis uses released outputs associated with repository snapshot `b566b08de2976451747215af4dc874a0846f6173`, and does not mix those results with the proposed approval/recovery contract. The stored outputs were reaggregated without changing inputs, labels, rules or runtime behaviour, so the reported figures describe that frozen snapshot rather than a later code state. A future rule, gate, policy or state-machine

Table 7. Principal tool routes and their current boundaries. A prompt instruction is not an authentication mechanism.

Tool	Purpose	Evaluated boundary
`review_parse`	Parse/resume input	Supported text and input digest
`review_check`	Run configured verifiers	Package validation; hard findings/hybrid candidates
`review_record_issue`	Register a judgement	Hard registration rejected; soft/invalid evidence pending; hybrid overlap gate
`review_status`	Show findings and unresolved items	Visibility does not force disposition
`review_decide`	Record disposition/final value	Name is recorded, not authenticated consent
`review_finalize`	Render, close and aggregate	Pending completion is not a reliable enforced prerequisite
`rules_list`, `rules_propose`, `memory_recall`	Inspect, edit/draft and recall	Tier-field restrictions do not cover all effective-authority changes

repair must be evaluated under a new frozen version; these historical detection scores do not certify it.

The artifact supplies a test suite, and a clean-environment full-suite reproduction is not established here: the checked environment encountered missing host-peer dependencies and a packaging-ignore assertion failure. We therefore do not assert an all-passing published test count. Moving to another document domain requires new predicates, source adapters and labelled evaluation, not merely a second YAML file.

4. Results

4.1. Retrospective Evaluation Setup

Corpus and labels.

The released generator (seed 20261029) produced 100 synthetic proposals and 253 planted defects over 23 verifiable rules: 98 absent, 37 negated, 52 placeholder and 66 keyword-present-but-empty (hollow). Thirteen documents have no planted defect; 87 have at least one. Expected verdicts are derived from the planted defects using the frozen policy: 30 Pass, 47 Revise and 23 Fail. Seventeen of the 30 policy-Pass documents contain permissible minor defects, so "policy Pass" must not be used as a synonym for "defect free". Thirty generated proposals were shaped on the verdict distribution of 30 historical expert opinions; these are not the original private documents or independent expert labels for the generated text.

The corpus and phrase pools were developed with the package. Its self-check guards selected execution-consistency conditions, including unexpected verifier failures on unplanted requirements, but does not require every planted defect to produce a verifier failure. Planted defects can remain pending or be missed. A generator-associated correction to the DOC-002 annex-number capture illustrates its value for development; it also means the corpus is developmental regression evidence, not an untouched semantic-review test set. The released labels are retained for this retrospective analysis, including the lifecycle and format ambiguities discussed in Section 3.3.

Configurations.

B0 uses rule-specific substrings without the structural and negation handling of the combined system, applying the same aggregation policy to its outputs. S-full uses the verifiers, candidate routes, gate and policy, with a deterministic *flag judge*: it proposes a finding when a candidate carries a negation, assertion-only or placeholder cue. It does not interpret design adequacy. S−verifier widens the hybrid statement search to the whole document. B0 does not invoke this judge; the comparison changes multiple mechanisms

and cannot isolate a causal typing effect. Memory is present in the software but these outputs do not measure a treatment effect of recalled feedback. No live LLM configuration was run for the reported corpus scores.

Metrics and analysis provenance.

Alignment is by (document, rule). We recomputed micro-precision, micro-recall and micro-F1 from summed true positives (TP), false positives (FP) and false negatives (FN), and mean per-document F1 from the stored outputs. An empty truth and empty prediction receives per-document F1 of one. We also recomputed policy-verdict accuracy and unweighted Cohen's κ [40,41], gate acceptance, and referral of missed planted defects to pending review. Out-of-package additions and soft-rule items do not enter the 23-rule detection scores. Published deterministic repetitions have identical core outputs and do not provide independent samples. The analysis reaggregates the stored records; it neither regenerates the corpus nor reruns the engine, and reported statistics are descriptive rather than inferential. The document is the observational unit; neither 253 defects nor 2300 rule–document pairs are independent document samples.

Table 8. Descriptive results from stored outputs on the fixed developmental corpus. No model capability, isolated typing effect or repaired approval contract is measured.

Configuration	TP	FP	FN	Precision	Recall	Micro-F1	Accuracy / κ
B0 keyword	47	124	206	0.275	0.186	0.222	28% / −0.043
S-full	168	0	85	1.000	0.664	0.798	85% / 0.763
S−verifier	168	0	85	1.000	0.664	0.798	85% / 0.763

4.2. Combined-Configuration Trade-offs (RQ2)

Micro-F1 rises from 0.2217 to 0.7981, a difference of 0.5764. Mean per-document F1 rises from 0.2161 to 0.7798, a paired mean difference of 0.5636. The earlier rounded 0.564 is therefore a different aggregation, not the micro-F1 difference. S-full has perfect per-document F1 on 42 documents and median F1 0.857. The improvement supports this combined configuration over this simple keyword baseline on shared developmental templates. It does not establish superiority over richer deterministic baselines, arbitrary prompted models or human review.

Table 9. Retrospective detection by operational tier and severity. Grouping identifies where errors occur; it does not establish that the tier caused the result.

	S-full			B0		
Group	TP / FP / FN	Precision	Recall	TP / FP / FN	Precision	Recall
Hard (10 rules)	125 / 0 / 17	1.000	0.880	37 / 16 / 105	0.698	0.261
Hybrid (13 rules)	43 / 0 / 68	1.000	0.387	10 / 108 / 101	0.085	0.090
Blocker	17 / 0 / 18	1.000	0.486	3 / 56 / 32	0.051	0.086
Major	99 / 0 / 42	1.000	0.702	22 / 53 / 119	0.293	0.156
Minor	52 / 0 / 25	1.000	0.675	22 / 15 / 55	0.595	0.286

Zero false positives is an observed property of these labels and this judge, not a consequence of located evidence. Eighteen of 35 blocker defects are missed. All blocker rules happen to be hybrid in this package; neither that composition nor the recall ordering proves a general severity–verifiability relationship. Seventeen misses occur in hard rules, showing that automatic eligibility can coexist with inadequate proxy predicates.

4.3. Traceability and Pending Workload (RQ3)

Sixty-eight of 85 missed planted defects occur in pending hybrid document–rule pairs (80% referral coverage). Seventeen remain unsurfaced by the evaluated automatic routes. Referral is not recovery: the stored result does not show that a reviewer read the item, identified the defect or corrected the verdict. The full pending set contains 932 unique hybrid pairs, mean 9.32 per document, median nine and range 6–13, excluding soft-rule items. Sixty-eight of these 932 pairs match planted defects (7.3% label density). The other pairs must not be called false positives: a pending item asserts uncertainty, not a defect. Nor is that density a measure of review time, which was not measured.

Table 10. Routes of planted defects. "Unsurfaced" refers to the evaluated machine routes, not to what a human could find by reading the document. Pending is not a confirmed finding.

Kind (planted count)	Found	Pending	Unsurfaced	Recall
Absent (98)	71	27	0	0.724
Negated (37)	31	6	0	0.838
Placeholder (52)	42	5	5	0.808
Hollow (66)	24	30	12	0.364
Total (253)	168	68	17	0.664

The flag judge submits 53 hybrid findings. Forty-two pass the gate; 11 are rejected, all for `QUOTE_TOO_SHORT`. Gate acceptance is 42/53 (79.2%). No absence, unlocated or out-of-candidate rejection occurs in these submissions, whose quotes originate in verifier-provided spans. That absence of other rejections does not validate resistance to misleading or fabricated model evidence. The 43 hybrid true positives comprise 42 gate-accepted submissions and one direct SEC-002 structural failure on `syn-rand-46`; describing all 43 as model judgements would misstate their provenance. Rejected true findings remain pending. Revising the quote floor or overlap predicate would require affected boundary tests and reevaluation, not simply transferring this result to a new gate.

4.4. Policy-Verdict Errors and Sensitivity

S-full and S−verifier are identical in these outputs. Widening the region therefore does not expose a difference for this cue-driven judge; it cannot establish the effect of scoping a generative judge. Table 11 shows the S-full recommendations against the policy-derived labels.

Table 11. S-full verdict confusion matrix; rows are expected and columns suggested. All 15 disagreements are under-calls.

Expected / suggested	Pass	Revise	Fail
Pass	30	0	0
Revise	6	41	0
Fail	4	5	14

Fifteen of 70 policy-non-Pass documents are under-called (21.4%); ten receive Pass (14.3%), comprising six expected Revise and four expected Fail. Of 23 expected Fail documents, four receive Pass (17.4%) and nine receive a non-Fail recommendation (39.1%). Ten of the 40 suggested Pass recommendations are incorrect against the policy labels (25%). The 13 genuinely zero-defect documents all have no finding. Seventeen further documents are policy Pass despite minor planted defects. These denominators prevent both inflating "clean-document" performance and mistaking all defects on a Pass document for a false clearance.

With expected labels held fixed under the shipped $\theta = 3$ policy, changing the recommendation's minor threshold gives Table 12. The plateau is conditional on this corpus and finding set, not general robustness or a reason to disregard missed blockers.

Table 12. Recommendation-threshold sensitivity against the fixed original labels, not against labels regenerated under each threshold.

θ	1	2	3 (shipped)	4	5
Accuracy	72%	82%	85%	85%	85%
Cohen's κ	0.534	0.713	0.763	0.763	0.763

4.5. A Historical Walkthrough and Its Label Boundaries

The generated proposal `syn-hist-30` concerns a balance-query capability opened to third-party channels. Its three gate-confirmed blockers are placeholder concurrency control (CAP-001), a negated binding-relation check (SEC-002), and placeholder exposure-risk assessment (SEC-004). A fourth confirmed finding is the hard diagram rule DOC-001, while MON-001 remains pending. The stored expected and suggested verdicts are both Fail, and per-document F1 is 0.889. The automated hybrid actor in this corpus case is the flag judge, not an LLM.

Two parts of this case cannot be interpreted as independent evidence of business defects. DOC-001 fails an image-only diagram under the frozen Mermaid predicate, without assessing what the image shows. MON-001 treats "New page tracking points must be added to count usage" as lacking an implementation statement. For a proposal before construction, that may instead be an implementation plan whose completeness needs review; lack of completed deployment is not automatically a proposal defect. We retain these historical labels to keep the analysis version-consistent, but identify the task/label discrepancies. Domain-owner adjudication and a new frozen evaluation are required before treating a corrected predicate as validated.

4.6. Development-Team Workflow Evidence (RQ4)

The authors report five runs on three private technical proposals: one revised/reviewed again and one resumption run. Only two reviews reached a final verdict. Across these runs they report 58 findings: 40 verifier-located, 12 model-judged and six operator additions; three of the 12 model-judged submissions were rejected for out-of-candidate quotations. They also report 18 decisions (15 accept, three dismiss, no rewrite), statistics for 13 rules and 21 exemplars. These categories describe the authors' workflow account, not a reconstructed private-store audit or an independent expert study. The private source documents, model/prompt configuration and full approval/decision trace are not available for independent verification here.

The account supports feasibility of recording and aggregating feedback in ordinary operator-led use. It does not establish verified consent for every record, guaranteed pending disposition, reduced workload, independent reviewer acceptance or a learning curve. It also does not repair the callable-interface and configuration-recovery counterexamples described in Section 3.7. An anonymous trace tying inputs, gate outcomes, dispositions and effective configuration to each final opinion would be needed to strengthen this workflow evidence without publishing confidential content.

5. Discussion

5.1. Answers to the Research Questions

RQ1: representation, operational authority, and implementation boundaries.

HARK makes review criteria inspectable as rule records with evidence expectations and operational routes. The package contains 10 hard, 13 hybrid, and five soft rules; all four blocker rules are hybrid. These tiers identify execution paths in this package, not intrinsic decidability of the corresponding business requirements. A successful lexical check establishes its observable predicate, not the adequacy of a load-testing plan or security design. The design separates verifier-produced findings, gate-confirmed findings, expert adoption, and final disposition. The evaluated snapshot implements parts of this separation but does not enforce the complete target authority contract: gate-confirmed blockers already affect the suggested verdict, and some hybrid structural failures are produced directly by verifiers.

RQ2: fixed-corpus detection and verdict trade-offs.

The combined S-full configuration produces 168 true positives, no false positives, and 85 false negatives, compared with 47, 124, and 206 for B0. Micro-F1 increases from 0.222 to 0.798, a difference of 0.576; the mean paired per-proposal F1 difference is 0.564, a distinct aggregation. This is a bounded improvement on a developmental synthetic regression corpus. B0 and S-full differ in detectors, cue handling, candidate routing, and evidence checks, and B0 does not use the flag judge. The comparison therefore does not isolate typing's causal effect. Identical S-full and S−verifier results show that this deterministic judge does not expose an effect of wider span scoping, not that scoping is effective or unnecessary for an LLM.

The verdict trade-off is one-sided but not benign. Agreement with policy-derived labels is 85%. Of 70 policy-non-Pass documents, 15 are under-called and 10 receive Pass: six expected Revise and four expected Fail. The latter four belong to a subset of 23 expected Fail documents. The corpus contains 13 zero-defect documents and 87 with at least one defect; the 30 policy-Pass documents are a different group, including 17 with minor defects. No finding is raised on the 13 zero-defect documents. These results describe the specified policy and labels, not reliable release decisions in operational use.

RQ3: traceability, gate acceptance, and unresolved workload.

The flag judge submits 53 hybrid findings, of which 42 pass the gate and 11 fail the quotation-length floor. Gate acceptance is 79.2%, not semantic evidence validity. The 43 hybrid true positives comprise those 42 accepted submissions plus one directly detected structural failure. The current location predicate checks overlap with a candidate span; neither overlap nor a genuine quotation establishes that the text supports the allegation. Zero false positives in this configuration are an empirical result, not a consequence of positional grounding.

Of 85 missed defects, 68 appear in pending review and 17 remain unsurfaced. The complete pending workload is 932 unique hybrid document–rule pairs, a mean of 9.32 per document and median of nine, excluding soft-rule items. Only 68 of these pairs match planted defects. This label density is not pending-item precision: pending expresses uncertainty, not an asserted defect. Referral coverage of 80% of misses provides no evidence that a reviewer read, resolved, or corrected them. A suggested Pass based on eligible findings is consequently not equivalent to a completed review.

RQ4: persistence, approval, and recovery.

The reported five development-team runs on three internal proposals, with two reaching a final verdict, provide workflow evidence. The reported 58 findings and subsequent statistics and exemplars describe recorded activity, not independently audited quality or learning gains. Decision records can be persisted and aggregated without establishing that their origin is authenticated or that they improve later reviews. The evaluated final-verdict path accepts a reviewer string, and instruction quotations for selected edits are length-checked rather than verified as human authorisation. Pending findings can remain when a review is finalised.

The authority and recovery gaps extend beyond tier editing. Changes to no-candidate behaviour, draft activation, finding severity, and reset can alter effective decision behaviour without changing a stored tier. The displayed `pack@ver+ovN` summary counts overridden rules rather than identifying their contents; different behaviours can share the same label. Authenticated approval bound to the action and effective content, control of all authority-changing operations, and immutable configuration snapshots are target contracts, not capabilities demonstrated by this snapshot. The observed counterexamples concern callable interfaces and do not establish malicious activity in the internal runs.

5.2. Implications for Evidence-Bearing Document Review

The useful abstraction is a review record linking a criterion, an evidence object, an operational route, and a disposition. This complements comment-generation approaches and evidence-aware review workflows [8,24]: it makes the inputs to a recommendation inspectable without treating inspectability as correctness. Human approval remains a design requirement whose engineering implementation must be distinguished from ordinary operator-led workflow.

The same abstraction could support assurance cases, compliance dossiers, or architecture designs if each supplies explicit criteria and addressable evidence. Such adaptation would require domain-specific evidence objects, lifecycle assumptions, rule provenance, and disposition responsibilities. This study validates none of those additional applications. Nor does narrow model input eliminate prompt injection or feedback-storage privacy risks [31,37]. The present evidence supports the architecture's executable composition and its fixed-corpus trade-offs, while exposing the limits that govern its interpretation.

6. Limitations and Future Work

Developmental synthetic evaluation.

The corpus reuses phrase pools and was developed alongside the rule package. Its planted labels and expected verdicts reflect the generator and the same policy used for evaluation; the self-check is an execution-consistency check that allows planted defects to remain pending or missed. The comparison is therefore a regression result, not an independent estimate of semantic review accuracy. Documents are the resampling unit; 253 defects and 2300 verifiable rule–document pairs are not independent document samples, and byte-identical deterministic repetitions add no sampling evidence. The reported statistics are descriptive, and no confidence intervals or significance tests are claimed. Generalisation would require frozen development artefacts and independently labelled, source-separated material.

Judge and component attribution.

The flag judge reacts to verifier cues rather than interpreting design intent. Its recall and precision are neither general lower nor upper bounds for an LLM, which could both recover semantic defects and introduce unsupported findings. The end-to-end comparison

changes multiple components and does not identify typing's individual effect. No reported experiment establishes cross-model stability, semantic entailment at the gate, or the benefit of span scoping for generative judges. Controlled component tests and model-based evaluations are possible extensions, not results of this study.

Human approval and effective authority.

The evaluated implementation assumes a trusted operator-led workflow; it does not authenticate human approval through its reviewer-name or instruction-quotation fields. Pending issues can survive finalisation. Existing tier restrictions do not cover the full effective-authority surface, including parameter changes, draft activation, finding-level severity, and reset. A stricter implementation would need approvals bound to a trusted principal, action, object, effective content, and consumption state, together with explicit disposition or waiver of unresolved items. These controls are planned and have not been implemented or validated here. Their absence limits assurance claims but does not imply that the internal runs involved misuse.

Version recovery and feedback use.

The override-count version label is a display summary, not a unique content identifier. It cannot alone reproduce a past review or distinguish different configurations with the same number of overridden rules. The target recovery contract would preserve the effective rules, lexicon, overrides, policy, parser and verifier versions, and an audit cursor in an immutable snapshot. Current persistence and aggregation evidence does not establish complete historical replay, authenticated attribution of every update, or net benefits from feedback. No longitudinal improvement claim is made.

Domain, lifecycle, and input coverage.

The 28-rule Chinese-language package concerns one organisation's technical proposal review. Hard denotes the snapshot's automatic route and may use a lexical or structural proxy narrower than the business criterion. Requiring Mermaid in the current package reflects a parser-visible check; an unreadable image does not by itself prove a defective design without a governing format requirement. Likewise, future-tense implementation plans must be judged against proposal-stage criteria rather than treated as failed implementation acceptance. Changing these predicates, labels, tiers, or evidence rules would require revalidation and, where affected, recomputation; the current metrics cannot be reassigned to a revised task. Other languages and document families require their own evidence mapping and validation.

Workflow evidence and human factors.

Three internal documents and five development-team runs, only two carried to a final verdict, are insufficient to establish external reviewer acceptance, reduced workload, or better decisions. The reported counts are not an independent audit of the private pilot store. Pending referral is not human resolution, and agreement between suggested and final verdicts is not a direct measure of rubber-stamping. Human disposition quality, time costs, missed pending items, and automation bias remain unmeasured. Evaluation of these outcomes must also distinguish provisional recommendation from review completion.

7. Conclusions

HARK represents rule-governed, evidence-bearing document review through explicit criteria, operational authority routes, located evidence, provisional findings, and decision records. Technical solution proposals are the sole empirical instantiation; assurance cases, compliance dossiers, and architecture designs remain potential applications. The system

contribution is the executable combination of these review objects and interfaces, not a guarantee that a model, verifier, or recorded operator name establishes an accountable final decision.

On the fixed synthetic regression corpus, the combined configuration achieves micro-F1 0.798 versus 0.222 for B0, with 168 true positives, no false positives, and 85 misses. Policy-verdict agreement is 85%; 15 of 70 policy-non-Pass documents are under-called, including 10 suggested Pass and four Fail-to-Pass cases among 23 expected Fail documents. Sixty-eight misses enter pending review, without demonstrated human resolution. The reported development-team runs support workflow and feedback persistence but not independent quality or online learning benefits. Authenticated approval, a complete effective-authority ceiling, and content-addressed recovery remain unimplemented target contracts. These boundaries define what the present design and validation evidence supports.

Author Contributions: Conceptualization, Z.Y.; methodology, Z.Y.; software, Z.Y.; validation, Z.Y., K.W. and Q.W.; formal analysis, Z.Y.; investigation, Z.Y.; resources, H.W.; data curation, Z.Y.; writing—original draft preparation, Z.Y.; writing—review and editing, all authors; visualization, Z.Y.; supervision, H.W.; project administration, Z.Y. All authors have read and agreed to the published version of the manuscript.
Funding: This research received no external funding.
Institutional Review Board Statement: Not applicable. The study analyses documents and workflow records produced in the routine internal review process of the authors' organisation. No human participants were recruited, no intervention was applied, and no personally identifiable information is reported.
Informed Consent Statement: Not applicable. The three internal proposals and the historical expert opinions are confidential organisational records and are reported only in aggregate; no individual participant data or identifiable personal information are included.
Data Availability Statement: The synthetic corpus, generator, judgement traces, evaluation scripts and rule package are available in the artifact repository at https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge, at snapshot `b566b08de2976451747215af4dc874a0846f6173`. An archived release is available at https://doi.org/10.5281/zenodo.23955205 (release v1.0.0, archived at commit `cbe1d33`, MIT licence). The corpus, ground-truth labels, manifest and stored evaluation runs in that archive are byte-identical to those of the cited snapshot, so the reported numbers can be recomputed from either. The cited snapshot adds three commits to the archived release—DOI wiring, the quoted-instruction check on authority-changing rule edits, and a release note—none of which alters the evaluation data. The historical expert opinions and the three internal proposals are private and cannot be released; the development-team workflow results are reported only in aggregate.
Generative AI Disclosure: Generative AI tools were used to assist structural revision, language editing, comparative-literature analysis, and cross-checking of the stored evaluation records. The editing environment was DeepSeek Harness (accessed November 2026) with DeepSeek models, used in model-assisted reviewer and revision roles. These tools did not generate experimental outcomes: every quantitative result derives from the stored deterministic runs described in Section 4. All suggested changes, quotations and references were checked against the source material by the authors, who take full responsibility for the content of this publication.
Conflicts of Interest: The authors declare no conflicts of interest. All authors are affiliated with the organisation whose technical proposal review provides the empirical setting of this study; the evaluation uses synthetic proposals and aggregated internal workflow records and reports no business outcomes.

References

1. Fagan, M.E. Design and Code Inspections to Reduce Errors in Program Development. *IBM Syst. J.* **1976**, *15*, 182–211. https://doi.org/10.1147/sj.153.0182
2. Laitenberger, O.; DeBaud, J.M. An Encompassing Life Cycle Centric Survey of Software Inspection. *J. Syst. Softw.* **2000**, *50*, 5–31. https://doi.org/10.1016/S0164-1212(99)00073-4
3. European Parliament and Council. Regulation (EU) 2024/1689 of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act). *Off. J. Eur. Union* **2024**, L 1689.
4. ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*; International Organization for Standardization: Geneva, Switzerland, 2018.
5. Génova, G.; Fuentes, J.M.; Llorens, J.; Hurtado, O.; Moreno, V. A Framework to Measure and Improve the Quality of Textual Requirements. *Requir. Eng.* **2013**, *18*, 25–41. https://doi.org/10.1007/s00766-011-0134-z
6. Femmer, H.; Méndez Fernández, D.; Wagner, S.; Eder, S. Rapid Quality Assurance with Requirements Smells. *J. Syst. Softw.* **2017**, *123*, 190–213. https://doi.org/10.1016/j.jss.2016.02.047
7. Chinnam, H. MARV: A Multi-Agent Approach for Validation of Software Requirements Specifications. Master's Thesis, Tampere University, Tampere, Finland, 2025. Available online: https://urn.fi/URN:NBN:fi:tuni-202507297873 (accessed on 30 October 2026).
8. Alsawalqah, H.; Abadleh, A.; Ibrahim, S.; Alqaralleh, B.A.Y.; Aoun, C.; Zreikat, A.I. Evidence-Aware Human-in-the-Loop LLM Review for Requirements-to-Planning Decisions. *Computers* **2026**, *15*, 608. https://doi.org/10.3390/computers15090608
9. Malik, S.; Singh, A.; Azad, A.P. PolicyGuard: From Organizational Policies to Neuro-Symbolic Compliance Review Engines. *arXiv* **2026**, arXiv:2606.32004.
10. Janisar, A.A.; Meidan, A.; Kalid, K.S.b.; Gilal, A.R.; Sarlan, A.B. Security Requirements Engineering: A Review and Analysis. *Computers* **2025**, *14*, 429. https://doi.org/10.3390/computers14100429
11. Zulfigarzada, S.; Gadirli, A.; Karimov, J.; Cerneckas, D.; Rackiene, R.; Azubalis, M. Cybersecurity Requirements and Certification Standards in Industrial Automation Systems: A Systematic Review. *Computers* **2026**, *15*, 364. https://doi.org/10.3390/computers15060364
12. Ferreira-Santos, J.; Pombo, L. The Art Nouveau Path: Requirements Engineering and Traceability for City-Scale In-the-Wild Mobile Augmented Reality Learning Services. *Computers* **2026**, *15*, 243. https://doi.org/10.3390/computers15040243
13. Veizaga, A.; Shin, S.Y.; Briand, L.C. Automated Smell Detection and Recommendation in Natural Language Requirements. *IEEE Trans. Softw. Eng.* **2024**, *50*, 695–720. https://doi.org/10.1109/TSE.2024.3361033
14. Wang, T.; Wang, W.; Yang, Y. AirReq: Automated Requirements Smell Detection and Elimination for Commercial Aircraft Systems. In Proceedings of the 2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW), 2025; pp. 173–180. https://doi.org/10.1109/REW66121.2025.00027
15. Hemmat, A.; Sharbaf, M.; Kolahdouz-Rahimi, S.; Lano, K.; Yassipour Tehrani, S. Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review. *Front. Comput. Sci.* **2025**, *7*, 1519437. https://doi.org/10.3389/fcomp.2025.1519437
16. Dolcetti, G.; Iotti, E. A Dual Perspective Review on Large Language Models and Code Verification. *Front. Comput. Sci.* **2025**, *7*, 1655469. https://doi.org/10.3389/fcomp.2025.1655469
17. Huang, L.; Yu, W.; Ma, W.; Zhong, W.; Feng, Z.; Wang, H.; et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* **2025**, *43*, 42. https://doi.org/10.1145/3703155
18. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; et al. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638
19. Huang, J.; Chen, X.; Mishra, S.; Zheng, H.S.; Yu, A.W.; Song, X.; et al. Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024), Vienna, Austria, 2024.

20. Djouvas, C.; Andreou, C.; Voutsa, M.C.; Tsapatsoulis, N. Model-Contingent Polarity Bias in Large Language Model Annotation: Implications for Semantic Multimedia Personalization. *Computers* **2026**, *15*, 262. https://doi.org/10.3390/computers15050262
21. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 1–45. https://doi.org/10.1145/3641289
22. Garcez, A.d'A.; Lamb, L.C. Neurosymbolic AI: The 3rd Wave. *Artif. Intell. Rev.* **2023**, *56*, 12387–12406. https://doi.org/10.1007/s10462-023-10448-w
23. Galitsky, B.; Rybalov, A. Neuro-Symbolic Verification for Preventing LLM Hallucinations in Process Control. *Processes* **2026**, *14*, 322. https://doi.org/10.3390/pr14020322
24. Pornprasit, C.; Tantithamthavorn, C. Fine-Tuning and Prompt Engineering for Large Language Models-Based Code Review Automation. *Inf. Softw. Technol.* **2024**, *175*, 107523. https://doi.org/10.1016/j.infsof.2024.107523
25. Sun, T.; Xu, J.; Li, Y.; Yan, Z.; Zhang, G.; Xie, L.; et al. BitsAI-CR: Automated Code Review via LLM in Practice. In Proceedings of the Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Trondheim, Norway, 2025. https://doi.org/10.1145/3696630.3728552
26. Tantithamthavorn, K.; Zou, Y.; Wong, A.; Gupta, M.; Wang, Z.; Buller, M.; et al. RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-Based Code Review Automation at Atlassian. In Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2026), 2026; pp. 34–45. https://doi.org/10.1145/3786583.3786851
27. Lin, H.Y.; Liang, M.; Tantithamthavorn, K.; Thongtanunam, P. Is Agentic Code Review Helpful? Mining Developers' Feedback to CodeRabbit Reviews in the Wild. *arXiv* **2026**, arXiv:2607.03316.
28. CodeRabbit. Learnings. CodeRabbit Documentation. Available online: https://docs.coderabbit.ai/knowledge-base/learnings (accessed on 5 November 2026).
29. Gao, T.; Yen, H.; Yu, J.; Chen, D. Enabling Large Language Models to Generate Text with Citations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore, 2023; pp. 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398
30. Zhang, J.; Chen, Y.; Commodore-Mensah, Y.; Oberst, M. Verifiable by Construction: Claim-Level Evaluation of Verbatim Citation in Clinical Question Answering. *arXiv* **2026**, arXiv:2609.15964.
31. Shi, J.; Yuan, Z.; Liu, Y.; Huang, Y.; Zhou, P.; Sun, L.; et al. Optimization-Based Prompt Injection Attack to LLM-as-a-Judge. In Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024), Salt Lake City, UT, USA, 2024; pp. 660–674. https://doi.org/10.1145/3658644.3690291
32. Mosqueira-Rey, E.; Hernández-Pereira, E.; Alonso-Ríos, D.; Bobes-Bascarán, J.; Fernández-Leal, Á. Human-in-the-Loop Machine Learning: A State of the Art. *Artif. Intell. Rev.* **2023**, *56*, 3005–3054. https://doi.org/10.1007/s10462-022-10246-w
33. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; et al. A Survey on Large Language Model Based Autonomous Agents. *Front. Comput. Sci.* **2024**, *18*, 186345. https://doi.org/10.1007/s11704-024-40231-1
34. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; et al. Self-Refine: Iterative Refinement with Self-Feedback. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
35. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
36. Zhang, J.; Hu, S.; Lu, C.; Lange, R.; Clune, J. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv* **2025**, arXiv:2505.22954.
37. Wang, B.; He, W.; Zeng, S.; Xiang, Z.; Xing, Y.; Tang, J.; et al. Unveiling Privacy Risks in LLM Agent Memory. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), Vienna, Austria, 2025; pp. 25241–25260. https://doi.org/10.18653/v1/2025.acl-long.1227

38. Mermaid: Generation of Diagrams and Flowcharts from Text. Available online: https://mermaid.js.org (accessed on 30 October 2026).
39. HARK: Human-Gated Authority-Typed Review Knowledge—Artifact Repository for This Paper. Available online: https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge (accessed on 4 November 2026).
40. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educ. Psychol. Meas.* **1960**, *20*, 37–46. https://doi.org/10.1177/001316446002000104
41. Landis, J.R.; Koch, G.G. The Measurement of Observer Agreement for Categorical Data. *Biometrics* **1977**, *33*, 159–174. https://doi.org/10.2307/2529310